Week 4

1. Determine if String Halves Are Alike

Program:

```
class Solution {
    public boolean halvesAreAlike(String s) {
        int mid = s.length() / 2;
        int countA = 0;
        int countB = 0;

        for (int i = 0; i < mid; i++) {
            if (isVowel(s.charAt(i))) {
                countA++;
            }
        }

        for (int i = mid; i < s.length(); I++) {
            if (isVowel(s.charAt(i))) {
                countB++;
            }
        }

        return countA == countB;
    }

    private boolean isVowel(char c) {
```

```
        return c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u'
            || c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U';
    }
}
```

Output:

Input

s ="book"

Output

true

Expected

True

2. Lapindromes:

Program:

```
import java.util.*;



class Codechef {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
```

```
int T = sc.nextInt();



while (T-- > 0) {

    String s = sc.next();



    int n = s.length();

    int mid = n / 2;



    String left = s.substring(0, mid);

    String right;



    if (n % 2 == 0) {

        right = s.substring(mid);

    } else {

        right = s.substring(mid + 1);
```

```
}


int[] count = new int[26];


// Count characters in left half
for (char c : left.toCharArray()) {
    count[c - 'a']++;
}


// Remove characters using right half
for (char c : right.toCharArray()) {
    count[c - 'a']--;
}


boolean lapindrome = true;
```

```
    for (int i = 0; i < 26; i++) {
        if (count[i] != 0) {
            lapindrome = false;
            break;
        }
    }

    if (lapindrome) {
        System.out.println("YES");
    } else {
        System.out.println("NO");
    }
}

sc.close();
```

```
    }

}
```

Output:

6

gaga

abcde

rotor

xyzxy

abbaab

ababc

3.Compare The Triplets

Program:

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {
```

```java
    public static List<Integer> compareTriplets(List<Integer> a,
List<Integer> b) {

        int alice = 0;
        int bob = 0;

        for (int i = 0; i < 3; i++) {
            if (a.get(i) > b.get(i)) {
                alice++;
            } else if (a.get(i) < b.get(i)) {
                bob++;
            }
        }

        List<Integer> result = new ArrayList<>();
        result.add(alice);
        result.add(bob);

        return result;
    }
}

    /*
     * Complete the 'compareTriplets' function below.
     *
     * The function is expected to return an INTEGER_ARRAY.
     * The function accepts following parameters:
     *  1. INTEGER_ARRAY a
     *  2. INTEGER_ARRAY b
     */









public class Solution {
    public static void main(String[] args) throws IOException {
```

```
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));

        List<Integer> a =
Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split(" "))
            .map(Integer::parseInt)
            .collect(toList());

        List<Integer> b =
Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split(" "))
            .map(Integer::parseInt)
            .collect(toList());

        List<Integer> result = Result.compareTriplets(a, b);

        bufferedWriter.write(
            result.stream()
                .map(Object::toString)
                .collect(joining(" "))
            + "\n"
        );

        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

Output:

Input (stdin)

```
5 6 7
3 6 10
```

Your Output (stdout)

```
1 1
```

Expected Output

```
1 1
```

4.Contains Duplicates

Program:

```
import java.util.*;

class Solution {
    public boolean containsDuplicate(int[] nums) {
        HashSet<Integer> set = new HashSet<>();

        for (int num : nums) {
            if (set.contains(num)) {
                return true;
            }

            set.add(num);
        }

        return false;
    }
}
```

Output:

```
Input
  nums =
  [1,2,3,1]
Output
  true
Expected
  true
```

5.Time Conversion

Program:

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
```

```java
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    /*
     * Complete the 'timeConversion' function below.
     *
     * The function is expected to return a STRING.
     * The function accepts STRING s as parameter.
     */

    public static String timeConversion(String s) {
    // Write your code here

        String period = s.substring(8, 10);
        int hour = Integer.parseInt(s.substring(0, 2));

        if (period.equals("AM")) {
            if (hour == 12) {
                hour = 0;
            }
        } else {
            if (hour != 12) {
                hour += 12;
            }
        }

        return String.format("%02d", hour) + s.substring(2, 8);
    }
}
```

```
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));

        String s = bufferedReader.readLine();

        String result = Result.timeConversion(s);

        bufferedWriter.write(result);
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

Output:

Input (stdin)

```
07:05:45PM
```

Your Output (stdout)

```
19:05:45
```

Expected Output

```
19:05:45
```

6.Move Zeroes

Program:

```
class Solution {
    public void moveZeroes(int[] nums) {
        int index = 0;

        // Move all non-zero elements to the front
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != 0) {
                nums[index] = nums[i];
                index++;
```

```
            }
        }

        // Fill the remaining positions with 0
        while (index < nums.length) {
            nums[index] = 0;
            index++;
        }
    }
}
```

Output:

Input

nums =

[0,1,0,3,12]

Output

[1,3,12,0,0]

Expected

[1,3,12,0,0]

7.Diagonal Difference

Program:

```
import java.io.*;
import java.util.*;
import java.util.stream.*;
import static java.util.stream.Collectors.toList;

class Result {

    /*
     * Complete the 'diagonalDifference' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts 2D_INTEGER_ARRAY arr as parameter.
     */

    public static int diagonalDifference(List<List<Integer>> arr) {

        int n = arr.size();
```

```
        int primaryDiagonal = 0;
        int secondaryDiagonal = 0;

        for (int i = 0; i < n; i++) {
            primaryDiagonal += arr.get(i).get(i);
            secondaryDiagonal += arr.get(i).get(n - 1 - i);
        }

        return Math.abs(primaryDiagonal - secondaryDiagonal);
    }
}

public class Solution {

    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader =
                new BufferedReader(new InputStreamReader(System.in));

        BufferedWriter bufferedWriter =
                new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int n = Integer.parseInt(bufferedReader.readLine().trim());

        List<List<Integer>> arr = new ArrayList<>();

        IntStream.range(0, n).forEach(i -> {
            try {
                arr.add(
                    Stream.of(
                        bufferedReader.readLine()
                            .replaceAll("\\s+$", "")
                            .split(" ")
                    )
                    .map(Integer::parseInt)
                    .collect(toList())
                );
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
```

```
        });

        int result = Result.diagonalDifference(arr);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

Output:

Input (stdin)

```
3
11 2 4
4 5 6
10 8 -12
```

Your Output (stdout)

```
15
```

Expected Output

```
15
```

8.Transpose Matrix

Program:

```
class Solution {
    public int[][] transpose(int[][] matrix) {
        int m = matrix.length;
        int n = matrix[0].length;

        int[][] result = new int[n][m];

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                result[j][i] = matrix[i][j];
            }
        }

        return result;
    }
}
```

Output:

```
Input
  matrix =
  [[1,2,3],[4,5,6],[7,8,9]]
Output
  [[1,4,7],[2,5,8],[3,6,9]]
Expected
  [[1,4,7],[2,5,8],[3,6,9]]
```

9.Multiply 2 Matrices

Program:

```
import java.util.*;

class Solution {

    public ArrayList<ArrayList<Integer>> multiply(int[][] mat1, int[][] mat2) {

        int n = mat1.length;

        ArrayList<ArrayList<Integer>> result = new ArrayList<>();

        for (int i = 0; i < n; i++) {

            ArrayList<Integer> row = new ArrayList<>();

            for (int j = 0; j < n; j++) {

                int sum = 0;

                for (int k = 0; k < n; k++) {
                    sum += mat1[i][k] * mat2[k][j];
                }

                row.add(sum);
            }

            result.add(row);
```

```
        }

        return result;
    }
}
```

Output:

Input:

n =

mat1[][] =

mat2[][] =

Your Output:

[[3, 3, 3], [3, 3, 3], [3, 3, 3]]

Expected Output:

[[3, 3, 3], [3, 3, 3], [3, 3, 3]]

10.Matrix block Sum

Program:

```
class Solution {
    public int[][] matrixBlockSum(int[][] mat, int k) {
        int m = mat.length;
        int n = mat[0].length;

        int[][] answer = new int[m][n];

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {

                int sum = 0;

                int r1 = Math.max(0, i - k);
                int r2 = Math.min(m - 1, i + k);
                int c1 = Math.max(0, j - k);
                int c2 = Math.min(n - 1, j + k);

                for (int r = r1; r <= r2; r++) {
```

```
                for (int c = c1; c <= c2; c++) {
                    sum += mat[r][c];
                }
            }

            answer[i][j] = sum;
        }
    }

    return answer;
  }
}
```

Output:

Input

```
mat =

[[1,2,3],[4,5,6],[7,8,9]]
k =

1
```

Output

```
[[12,21,16],[27,45,33],[24,39,28]]
```

Expected

```
[[12,21,16],[27,45,33],[24,39,28]]
```

11.Matrix Layer Rotation

Program:

```
import java.io.*;

import java.util.*;

import java.util.stream.*;

import static java.util.stream.Collectors.toList;


class Result {


    /*
```

```
 * Complete the 'matrixRotation' function below.
 *
 * The function accepts following parameters:
 *  1. 2D_INTEGER_ARRAY matrix
 *  2. INTEGER r
 */

public static void matrixRotation(List<List<Integer>> matrix, int r) {

    int m = matrix.size();
    int n = matrix.get(0).size();

    int layers = Math.min(m, n) / 2;

    for (int layer = 0; layer < layers; layer++) {

        int top = layer;
        int bottom = m - 1 - layer;
        int left = layer;
        int right = n - 1 - layer;

        List<Integer> elements = new ArrayList<>();

        // Top row
        for (int j = left; j <= right; j++) {
            elements.add(matrix.get(top).get(j));
        }
```

```
// Right column
for (int i = top + 1; i <= bottom; i++) {
    elements.add(matrix.get(i).get(right));
}

// Bottom row
for (int j = right - 1; j >= left; j--) {
    elements.add(matrix.get(bottom).get(j));
}

// Left column
for (int i = bottom - 1; i > top; i--) {
    elements.add(matrix.get(i).get(left));
}

int length = elements.size();
int rotation = r % length;

int index = rotation;

// Put rotated elements back

// Top row
for (int j = left; j <= right; j++) {
    matrix.get(top).set(j, elements.get(index));
    index = (index + 1) % length;
```

```
    }

    // Right column
    for (int i = top + 1; i <= bottom; i++) {
        matrix.get(i).set(right, elements.get(index));
        index = (index + 1) % length;
    }

    // Bottom row
    for (int j = right - 1; j >= left; j--) {
        matrix.get(bottom).set(j, elements.get(index));
        index = (index + 1) % length;
    }

    // Left column
    for (int i = bottom - 1; i > top; i--) {
        matrix.get(i).set(left, elements.get(index));
        index = (index + 1) % length;
    }
}

// Print rotated matrix
for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        System.out.print(matrix.get(i).get(j));

        if (j < n - 1) {
```

```
                System.out.print(" ");
            }
        }
        System.out.println();
    }
  }
}

public class Solution {

    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader =
                new BufferedReader(new InputStreamReader(System.in));

        String[] firstMultipleInput =
                bufferedReader.readLine()
                        .replaceAll("\\s+$", "")
                        .split(" ");

        int m = Integer.parseInt(firstMultipleInput[0]);
        int n = Integer.parseInt(firstMultipleInput[1]);
        int r = Integer.parseInt(firstMultipleInput[2]);

        List<List<Integer>> matrix = new ArrayList<>();

        IntStream.range(0, m).forEach(i -> {
```

```
        try {
            matrix.add(
                Stream.of(
                    bufferedReader.readLine()
                        .replaceAll("\\s+$", "")
                        .split(" ")
                )
                .map(Integer::parseInt)
                .collect(toList())
            );
        } catch (IOException ex) {
            throw new RuntimeException(ex);
        }
    });

    Result.matrixRotation(matrix, r);

    bufferedReader.close();
  }
}
```

Output:

Input (stdin)

```
4 4 1
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
```

Your Output (stdout)

```
2 3 4 8
1 7 11 12
```

```
5 6 10 16
9 13 14 15
```

Expected Output

```
2 3 4 8
1 7 11 12
5 6 10 16
9 13 14 15
```